

Небезопасные сторонние компоненты

Описание проблемы

Сторонние компоненты, используемые в проектах, могут содержать уязвимости. Поэтому важно [согласовывать их использование](#) с отделом Платформы и [поддерживать их в актуальном состоянии](#). Сторонние компоненты бывают разнообразными, и в основном делятся на:

- Библиотеки/пакеты
- Докер-образы
- Программы/инструменты

Обязательным требованием является расположение сторонних библиотек в специализированных реестрах или одной из групп репозиторий:

- <http://repo-dev.mgmt.tensor.ru/> - реестр npm-пакетов, python-библиотек, и бинарных файлов
- <http://maven.sbis.ru/> - реестр андроид-пакетов
- <https://git.sbis.ru/third-party> - для библиотек вне реестров
- <https://dev-image-store.tensor.ru/> - реестр докер-образов



Нельзя хранить и подтягивать при сборке стороннюю библиотеку просто из репозитория своего проекта.

Вероятно, вы уязвимы, если:

- **Вам неизвестны версии всех используемых компонентов** (как на стороне клиента, так и на стороне сервера). Сюда входят компоненты, которые вы используете непосредственно, а также вложенные зависимости.
- **Программное обеспечение уязвимо, не поддерживается или устарело.** Это касается ОС, сервера веб-приложений, системы управления базами данных (СУБД), приложений, API и всех компонентов, среды выполнения и библиотек.
- **Вы используете компоненты из ненадежных источников** или не проводите проверку их целостности.
- **Вы не выполняете валидацию входных данных** перед их передачей сторонним библиотекам, что может привести к эксплуатации уязвимостей, связанных с недостаточной проверкой входных данных.

Последствия эксплуатации

Использование небезопасных компонентов может привести ко множеству негативных последствий. Конкретные риски зависят от конкретной библиотеки, ее версии и известных уязвимостей.

Это может включать:

- Выполнение злоумышленником произвольного кода на серверах компании
- Отказ в обслуживании
- Атаки на пользователей и клиентов
- Утечку конфиденциальных данных

Примеры небезопасных компонентов

GhostScript, ImageMagick, ffmpeg

GhostScript, **ImageMagick** и **ffmpeg** — инструменты, используемые в наших проектах. Однако их использование может сопровождаться определенными проблемами безопасности, которые требуют внимания и мер предосторожности.

GhostScript: Уязвимости в GhostScript часто связаны с обработкой файлов PostScript и PDF, что может привести к выполнению произвольного кода на сервере.



В 2018 году была обнаружена критическая уязвимость в GhostScript (CVE-2018-16509), которая позволяла

злоумышленникам выполнять произвольный код на сервере через специально подготовленные PDF или PostScript файлы. Эксплуатация этой уязвимости могла привести к компрометации системы, предоставляя злоумышленникам возможность получить контроль над сервером, украсть данные или использовать ресурсы сервера в злонамеренных целях.



Новая уязвимость CVE-2024-29510, обнаруженная в Ghostscript, затрагивает версии до 10.03.0. Она позволяет обойти песочницу -dSAFER, что открывает доступ к опасным операциям, включая выполнение команд и файловый ввод-вывод. Проблема связана с устройством uniprint, которое формирует командные данные для принтеров.

Уязвимость угрожает веб-приложениям и сервисам для конвертации и предварительного просмотра документов, использующим Ghostscript. Злоумышленники уже используют этот баг в реальных атаках, маскируя файлы EPS под JPG.

Уязвимость исправлена в Ghostscript версии 10.03.1.

ImageMagick: Библиотека часто используется для обработки изображений, но уязвимости, такие как ImageTragick, позволяют злоумышленникам выполнять команды на сервере через специально подготовленные файлы.



Уязвимость ImageTragick (CVE-2016-3714) в ImageMagick позволяла злоумышленникам выполнять команды на сервере при обработке вредоносных изображений. Эксплуатация этой уязвимости могла привести к выполнению произвольного кода на сервере, что угрожало целостности данных, конфиденциальности информации и безопасности системы в целом.

ffmpeg: Уязвимости в обработке мультимедийных файлов могут позволить злоумышленникам выполнять произвольный код, используя специально подготовленные видео- или аудиофайлы.



Уязвимости в ffmpeg (CVE-2016-1897, CVE-2016-1898) позволяли злоумышленникам использовать специально подготовленные мультимедийные файлы для выполнения произвольного кода на сервере. Это могло привести к утечке данных, полному контролю над системой злоумышленниками, или использованию серверов для проведения дальнейших атак, включая участие в ботнетах.

Log4Shell

Уязвимость в библиотеке Log4j, используемой во множестве продуктов на Java. Эта критическая уязвимость позволяет злоумышленнику удалённое исполнение кода, просто отправив команду на логирование заранее заготовленной строки. При обнаружении уязвимости, ей были подвержены все версии библиотеки Log4j, что усложнило её исправление. Но новейшие версии библиотеки по умолчанию обеспечивают необходимую защиту.

Докер-образы

Обычно уязвимости в образах появляются из компонентов, которые в них включены. Такие как linux-пакеты, или другие образы. Просканировать образы на наличие подобных уязвимостей можно с помощью автоматизированного инструмента: <https://github.com/aquasecurity/trivy>. Не все уязвимости могут затрагивать используемый функционал, но всё равно стоит быть внимательным, и не допускать появления уязвимых компонентов.

Программные закладки

Стало актуально в последние годы - программное обеспечение может перестать работать по региональному признаку, или привести деструктивное поведение. Например, подключение библиотеки sweetalert2 могло повлечь нарушение функциональности: <https://github.com/sweetalert2/sweetalert2/releases/tag/v11.4.9>.

Как исправить?

Общие рекомендации

- [Согласовывайте](#) использование библиотеки с Платформой (дополнительно публикуйте новость на всю разработку).
- Регулярно обновляйте сторонние компоненты, чтобы закрыть известные уязвимости.

- **Удалите неиспользуемые зависимости, ненужные функции, компоненты, файлы и документацию.**
- **Минимизируйте функциональность.** Настройте библиотеки таким образом, чтобы они выполняли только необходимые для вашего приложения функции. Отключите все ненужные возможности, которые, потенциально, могут представлять угрозу безопасности.
- **Располагайте сторонние библиотеки только в соответствующих реестрах.**

GhostScript

В Ghostscript существует флаг `-dSAFER`, который используется для включения безопасного режима работы программы. Этот режим ограничивает возможности скриптов, выполняемых в процессе обработки файлов, что предотвращает выполнение потенциально опасных операций, таких как:

1. Доступ к файловой системе (чтение, запись или удаление файлов).
2. Вызов команд операционной системы.
3. Манипуляции с сетевыми ресурсами.

Использование флага `-dSAFER` особенно важно при обработке подозрительных или неизвестных файлов, таких как PDF-документы, PostScript-файлы и т.п., которые могут содержать вредоносный код. Включение безопасного режима позволяет минимизировать риски, связанные с выполнением вредоносных действий.



Ghostscript, начиная с версии 9.50 по умолчанию запускается с флагом `-dSAFER`.